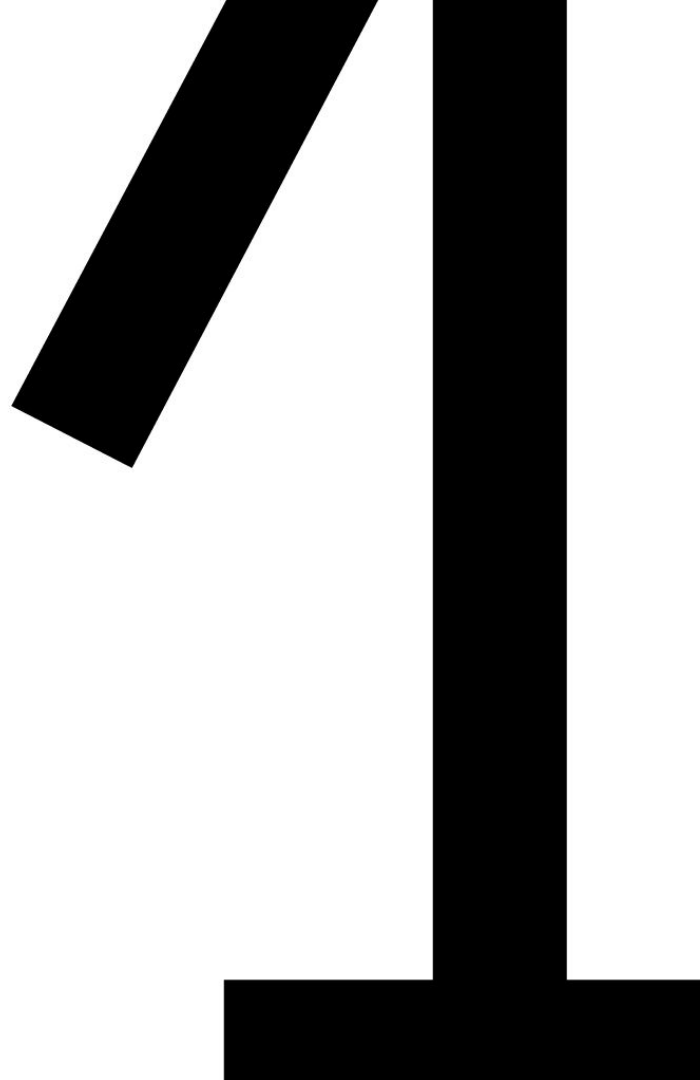


Design for change

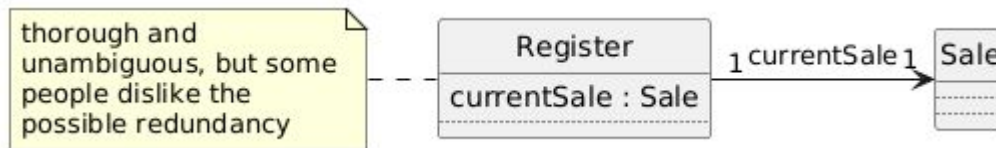
Visibility, dependency, GRASP, SOLID and more

Visibility



Definition of object visibility

- **Object visibility = the ability of an object to know or refer to another object**
- Until now visibility only by keeping a reference (pointer) in an attribute
- ... but there are other ways to have visibility.



Types of visibility (navigability)

- **Permanent visibility**
 1. **attribute visibility**
 2. **global visibility**
- **Temporary visibility**
 3. **visibility parameter**
 4. **local visibility**

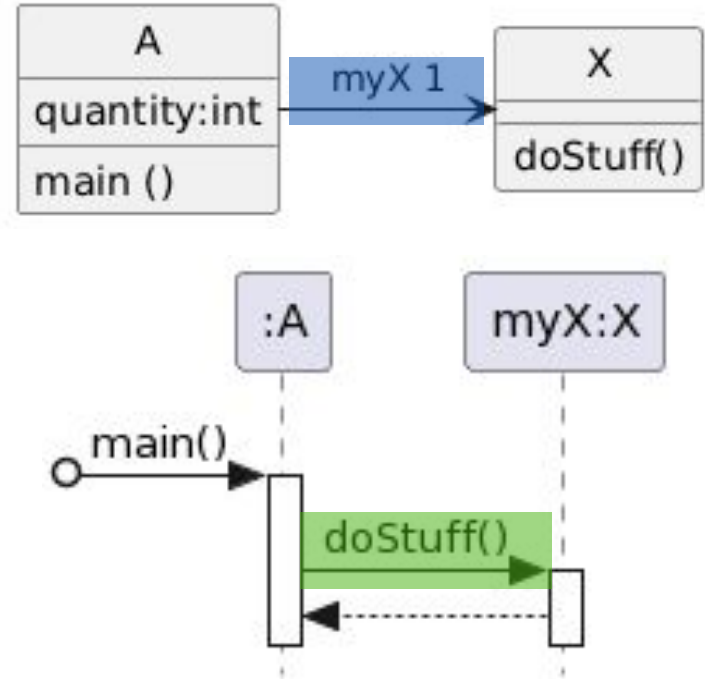


**Permanent visibility:
attributed**

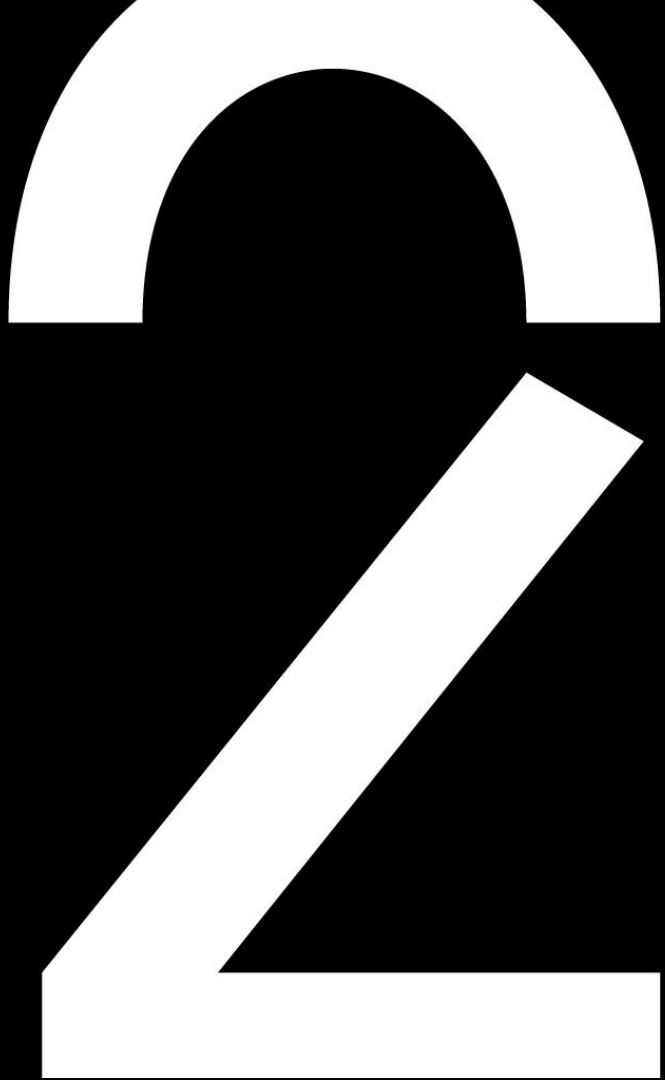
Permanent visibility: attribute

```
public class A {  
    private X myX;  
    private int number;  
  
    public static void main() {  
        myX.dostuff();  
    }  
}
```

```
public class X {  
    void dostuff(){}  
}
```



**Permanent visibility:
global**

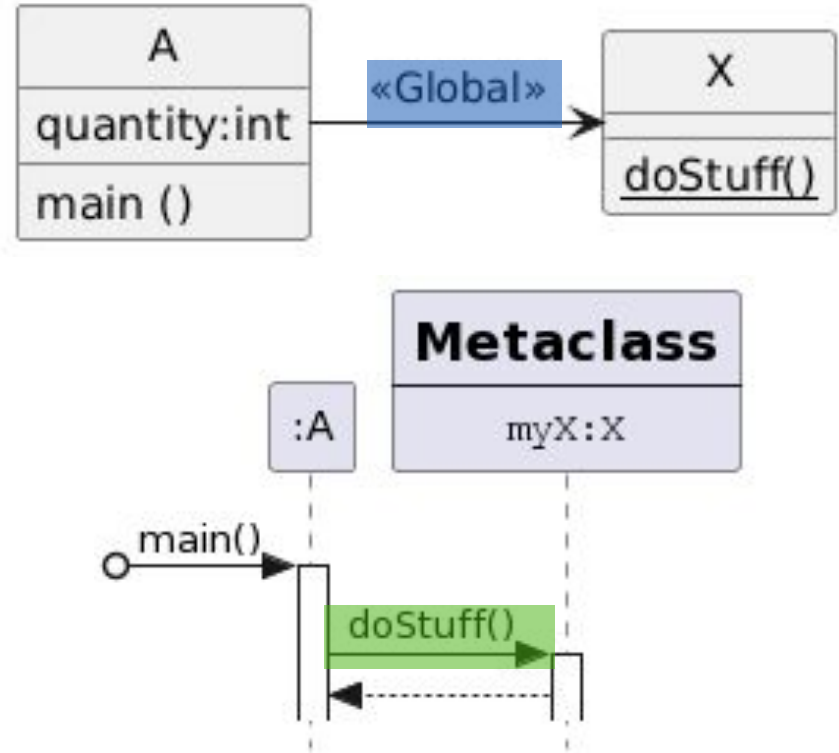


Permanent visibility: Global

```
import X
```

```
public class A{  
    public static void main() {  
        X.doStuff();  
    }  
}
```

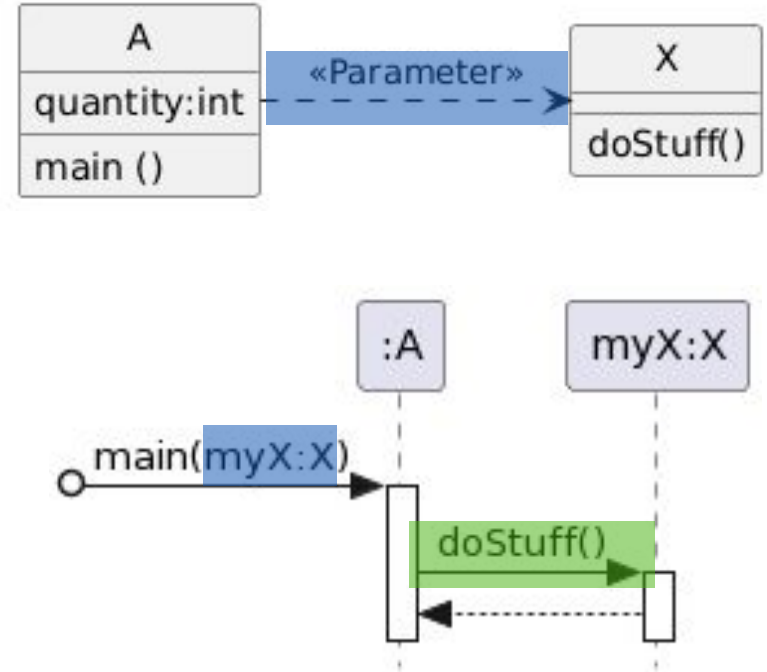
```
public class X{  
    public static void doStuff() {  
        //Do some stuff  
    }  
}
```



Temporary visibility: parameter

Temporary visibility: parameter

```
public class A {  
    private int number;  
  
    public static void main(X myX) {  
        myX.dostuff();  
    }  
}
```

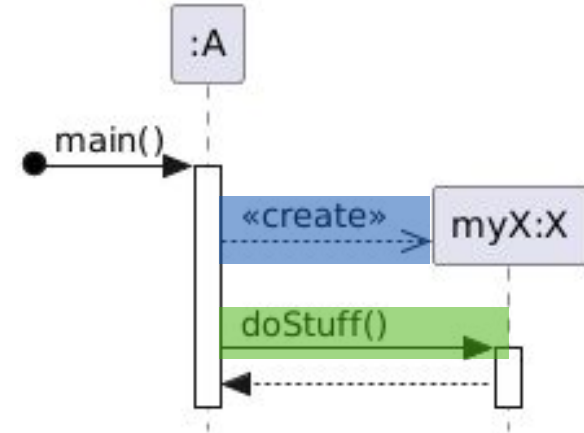
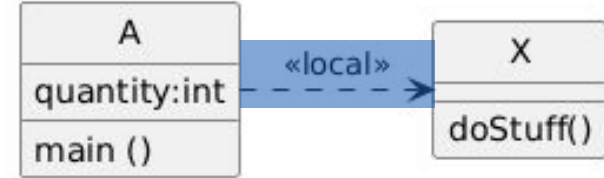


Temporary visibility: local



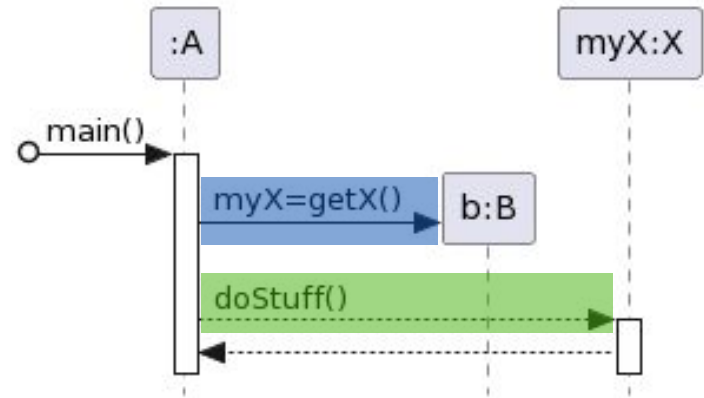
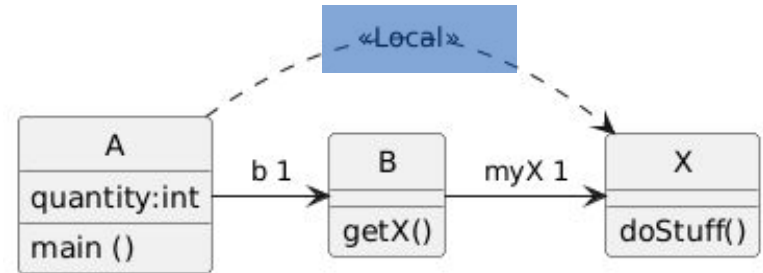
Temporary visibility: local 1/2

```
public class A {  
    private int number;  
  
    public static void main() {  
        X myX = new X();  
        myX.dostuff();  
    }  
}  
  
public class X {  
    public void dostuff();  
}
```

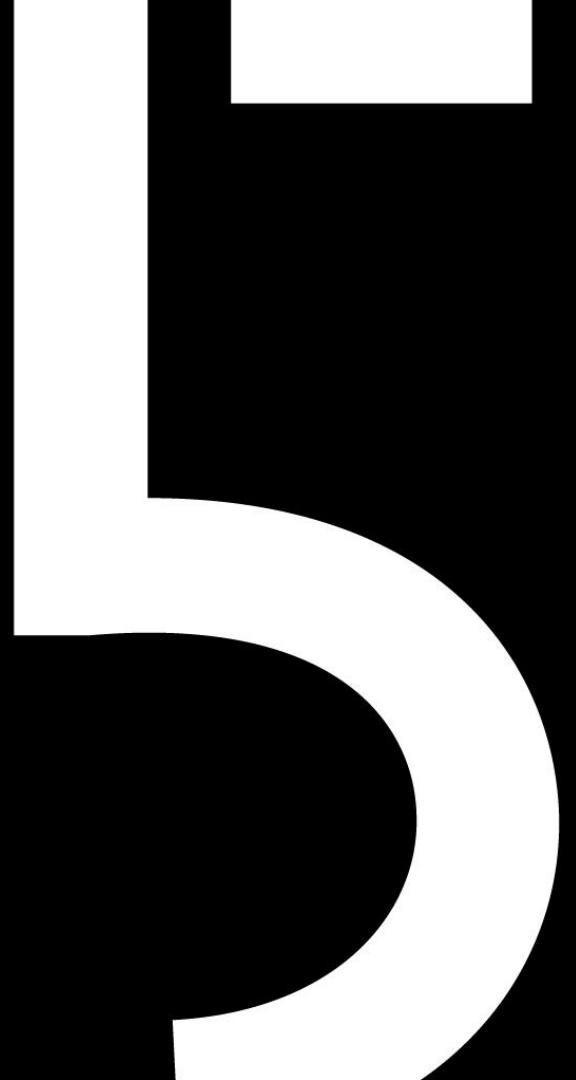


Temporary visibility: local 2/2

```
public class A {  
    private int number;  
    private B b;  
  
    public static void main() {  
        X myX = b.getX();  
        myX.dostuff();  
    }  
}
```

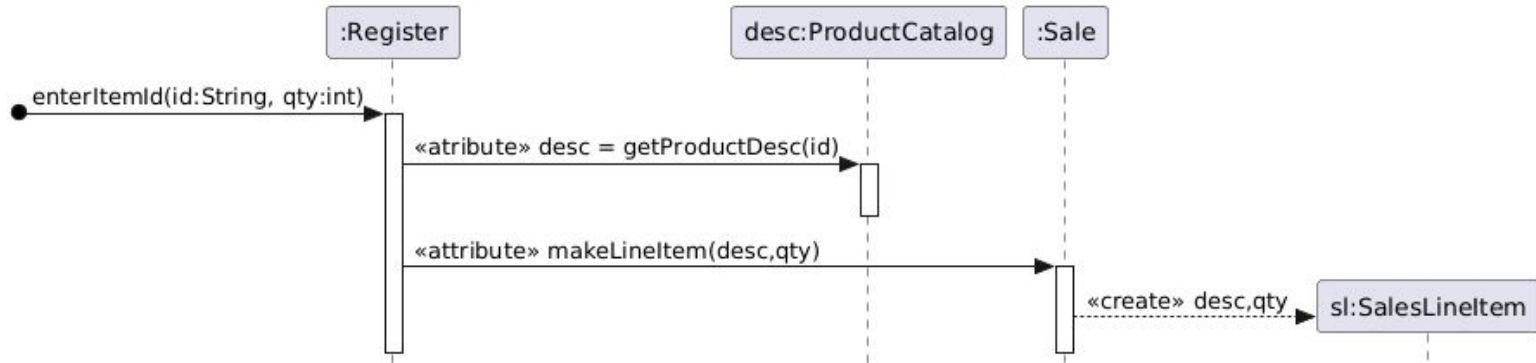


**Transforming temporary
visibility into attribute
visibility**



Transforming temporary visibility to attribute visibility

- Store a temporary variable in an attribute
 - Example
 - B can be assigned to A by passing it as a parameter in a method (often the constructor) of A and storing it in an attribute
- B enters A as a parameter but transforms to attribute when B is assigned to an attribute of A.(see message 2.1: desc at sl)*



Conclusion: visibility & linking



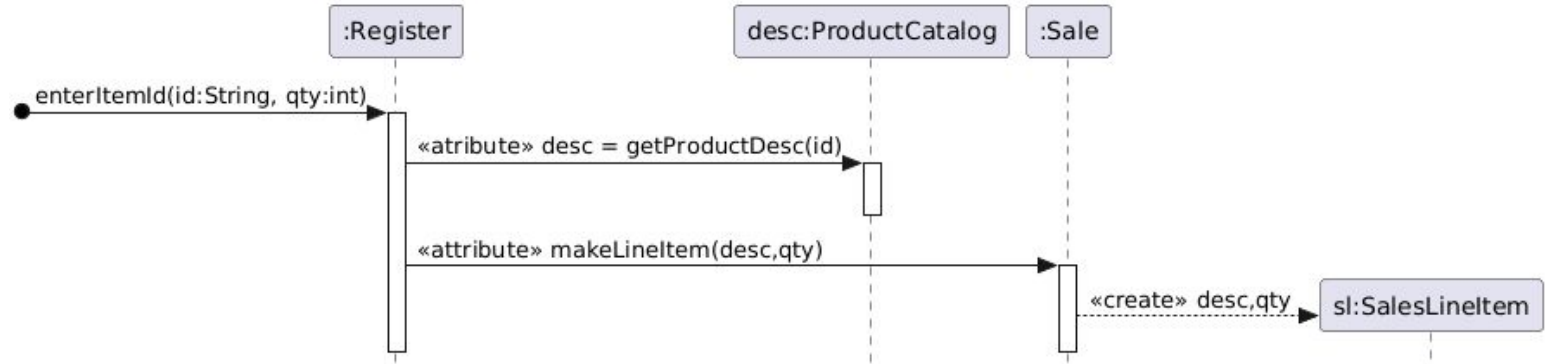
Conclusion: visibility & linking

- **Linking is not only determined by permanent visibility (associations) between classes**
 - also consider links between objects on interaction diagrams and temporary visibility
 - **inheritance also implies visibility on everything that is not private in the superclass and is thus a very strong coupling**
- Keep this in mind when you
 - create a design with classes with low coupling (*see later*)
 - your design subdivided into packages, which are interlinked in layers

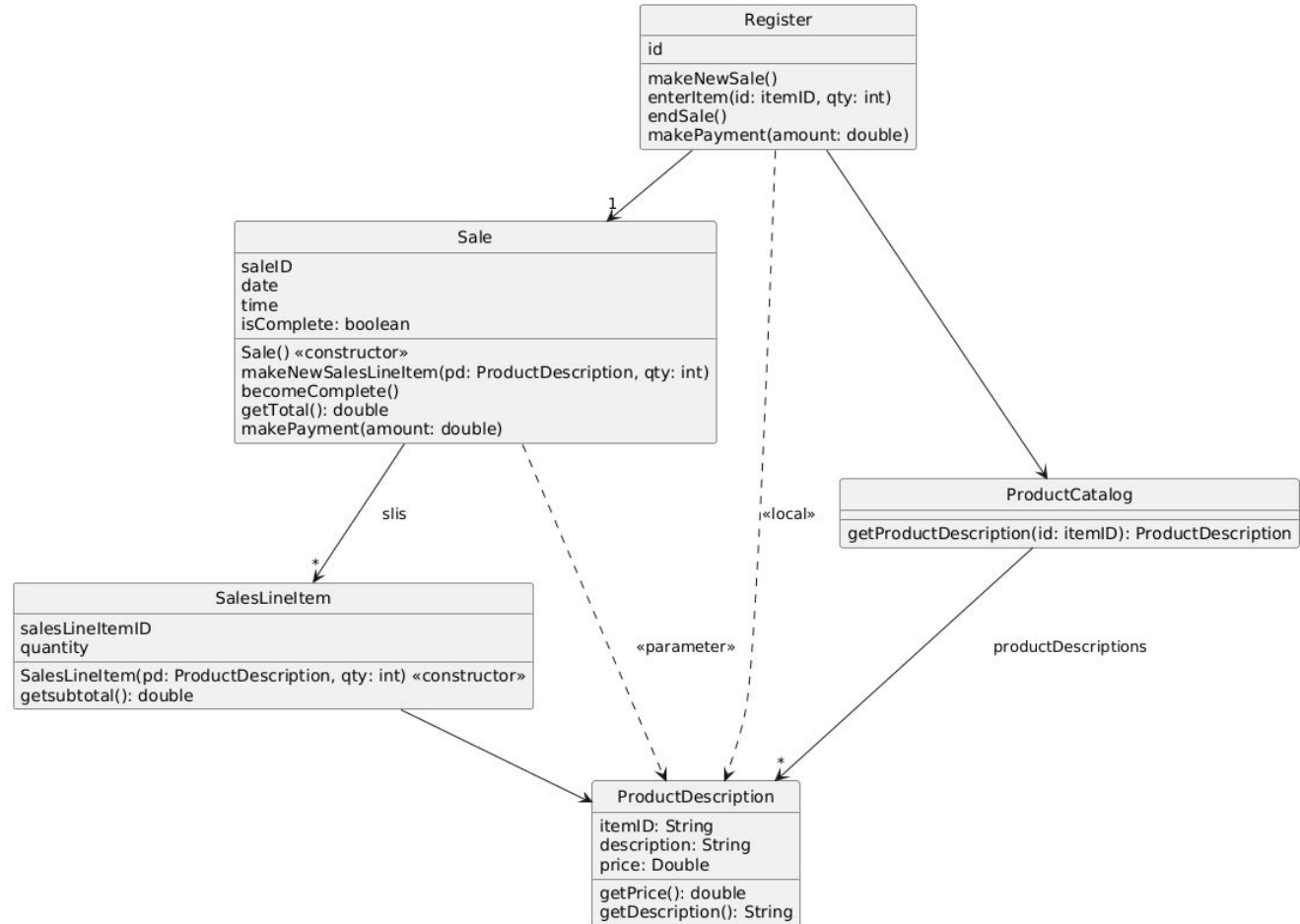
Summary: Visibility in practice

-
- Check that your interaction diagram is consistent with the visibilities in your design!
 - Go through interaction diagrams and determine for each message how the sender gets the reference to the receiver
 - Possibilities, in order of most used:
 - <<attribute>> The reference to an object is permanently known as a reference stored in an attribute of the sender.
 - <<parameter>> The reference to an object is temporarily known because this reference was passed along as a parameter when calling an operation in the transmitter.
 - <<local>> The reference to an object is temporarily stored as a local variable in the sender. For example, the sender calling the constructor of the receiver. The sender then knows the reference via a variable
 - <<global>> The referenced object is permanently globally available to the whole system. Avoid this high visibility and linking because detrimental!

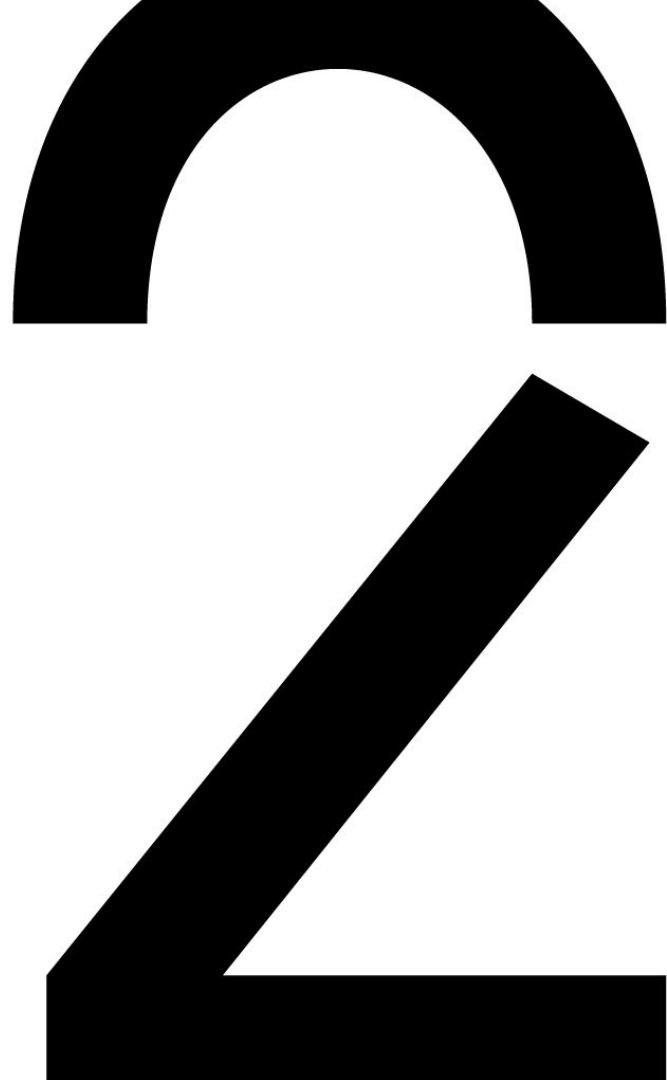
Visibility in an SD



Visibility in a DCD



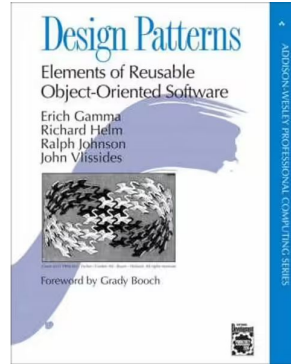
Patterns



Pattern families

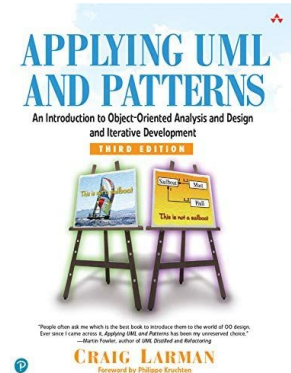
Gang of Four (GoF) design patterns - Gamma et al.
→ 23 patterns

-> in next courses

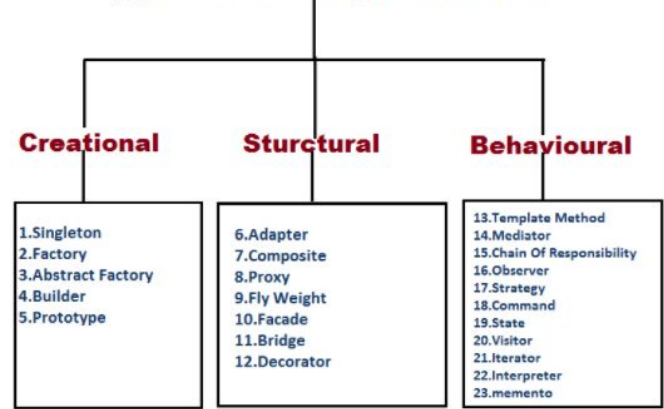


GRASP: General responsibility assignment software patterns or principles (Larman) → 9 patterns

-> 5 patterns in this course



Types Of Design Patterns



Why patterns?

Software patterns/principles for overall responsibility allocation

- Determine which class/method is responsible for implementing which code
- Does it occur only sporadically: to which class should this responsibility be assigned?

GRASP are good design solutions to common design problems

- As a novice developer, you can use it as a design, but ...
- the intention is to learn from them

They are principles.

- Once you understand the principles on which the patterns are based, you can apply them in all situations

So: "A set of patterns and principles that, if followed correctly, provide well-structured, maintainable code."

Intro GRASP

GRASP

GRASP patterns in Larman chapter 17

Low Coupling (evaluative basic principle)

High Cohesion (evaluative basic principle)

Controller

Creator

Information Expert

Other GRASP patterns

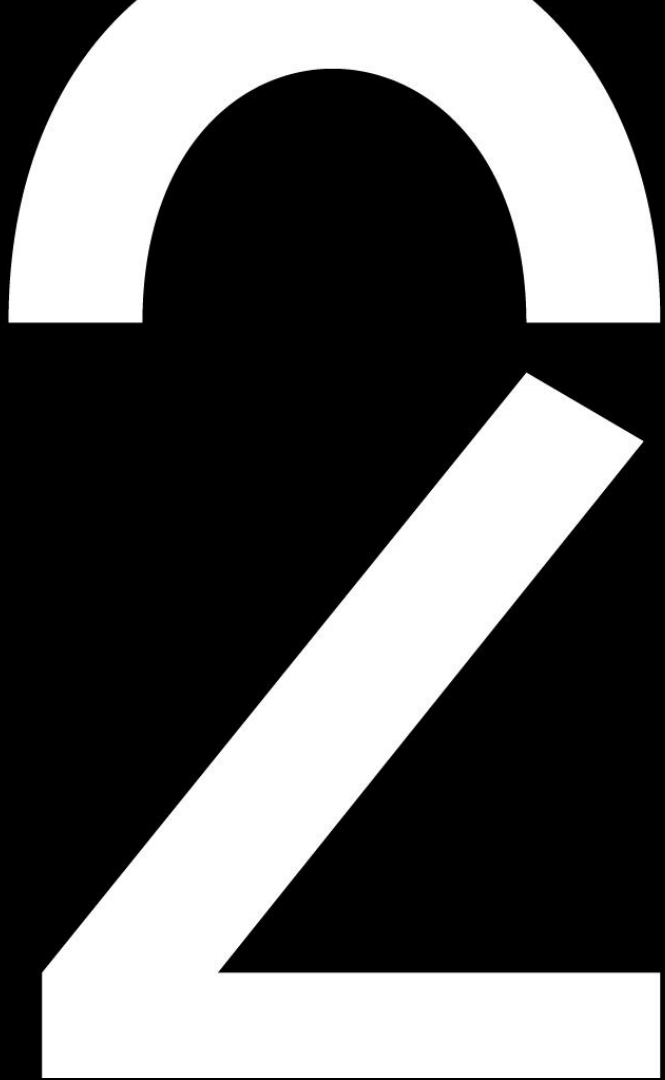
Polymorphism

Pure fabrication

Indirection

Protected variations

Basics
Low coupling &
high cohesion



Low coupling & high cohesion

- Low coupling: limit dependencies between objects to the absolute minimum
- High Cohesion: Ensure maximum coherence of responsibilities within a class



Principle: Low coupling

- Problem:
How do we ensure low dependency ("coupling"), low impact on changes and high reusability?
- Solution:
Assign responsibilities so that dependencies remain low.



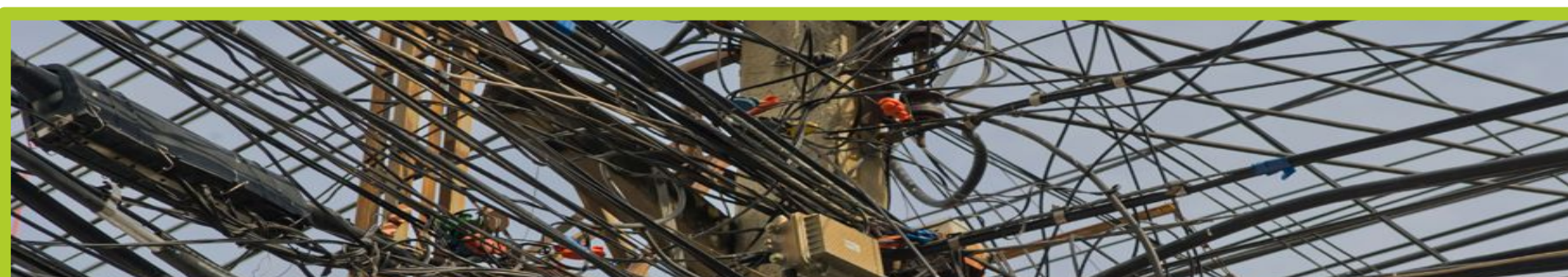
Principle: Low coupling

- Low coupling
 - Coupling is the extent to which an element is connected to, has knowledge of or relies on other elements.
 - Elements (such as classes) may be "too linked" with other elements due to:
 - Changes in one class lead to changes in many other classes
 - Reuse becomes more difficult due to dependencies



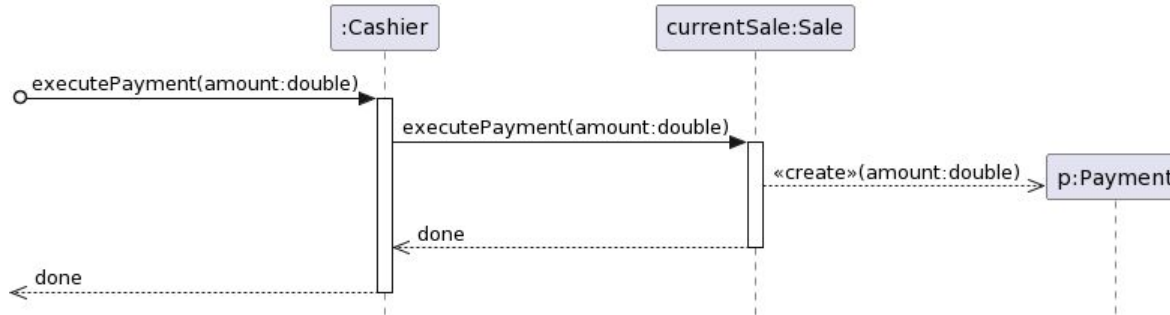
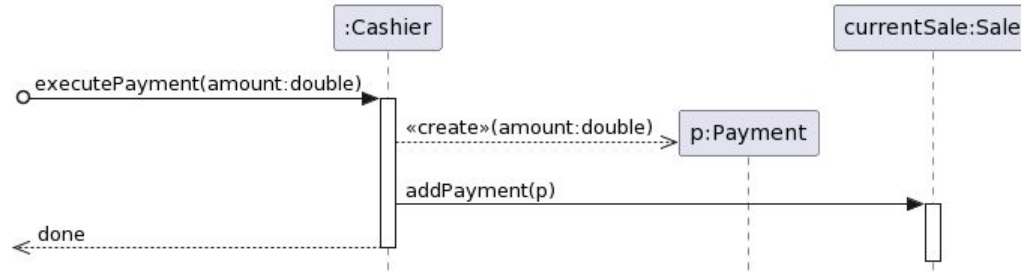
Principle: Low coupling

- Coupling is the degree to which classes depend on each other.
- This dependence is the result of several situations:
 - Class A has an attribute that refers to an object of Class B = **attribute coupling**
 - Class A object calls methods of a Class B object = **method coupling**
 - Class A has a method that references objects of Class B. This can be done by a local variable within the method (= **variable coupling**) or by a parameter of the method being an object of Class B = **parameter coupling**
 - Class A is a direct or indirect subclass of Class B = **inheritance coupling**
 - Class B is an interface and Class A implements this interface = **interface coupling**
- Coupling is determined between two classes, but more frequently the overall coupling of a (sub)system is considered



Example: Low coupling

- Example: which of the two systems has the lowest coupling and is preferred by the Low Coupling pattern?



Example: Low coupling

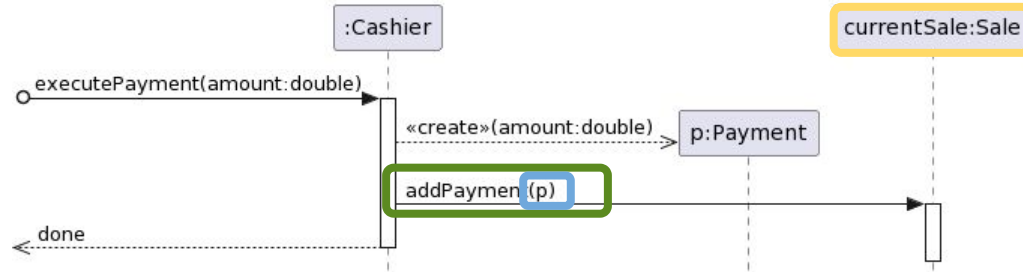
- Example: which of the two systems has the lowest coupling and is preferred by the Low Coupling pattern?

Total coupling:

1 x attribute coupling

1 x method coupling

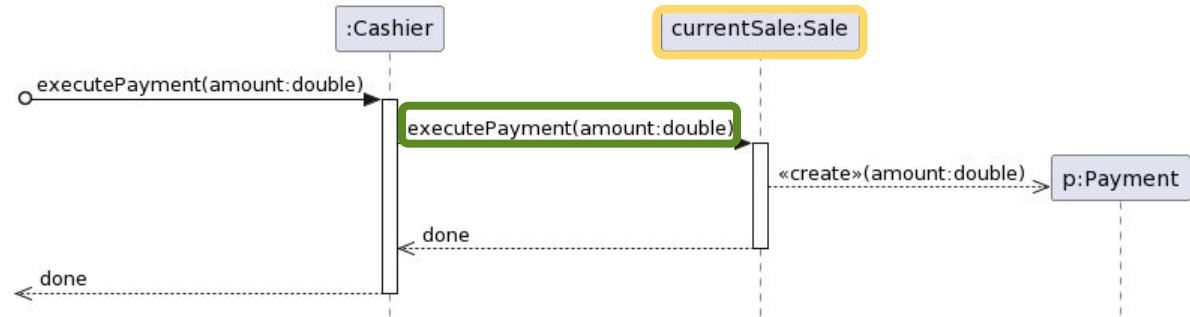
1 x parameter coupling



Total coupling:

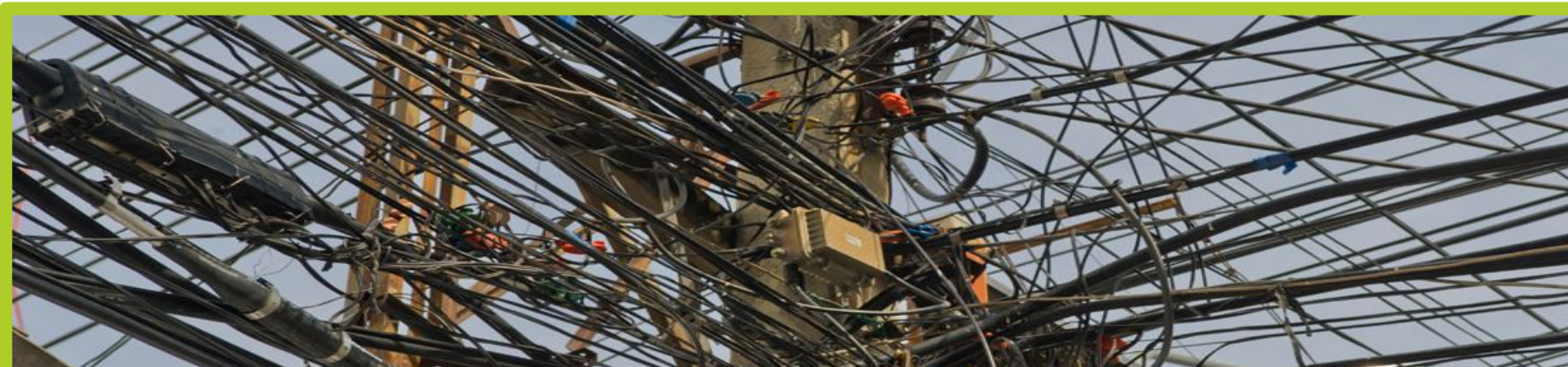
1 x attribute coupling

1 x method coupling



Principle: Low coupling

- Points of interest:
 - Dependencies in themselves are a necessity and not a problem. But we need to limit them so there are no negative effects associated with high dependencies:
 - e.g. Generalisation/acquisition
 - Absence of dependencies (putting much/everything into one god class) sins against the principle of object modelling and creates "low cohesion"
 - High coupling with highly stable elements is no problem (Java SDK)



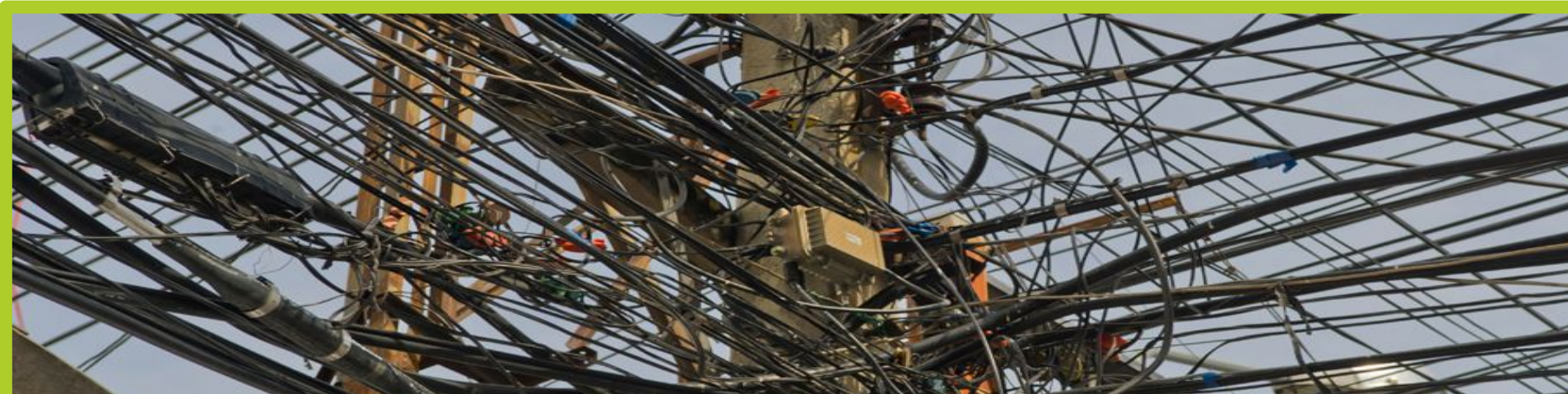
Principle: high cohesion

- High cohesion
 - Cohesion is to what extent the responsibilities of an element are related or focused.
 - Elements (such as classes) can have "too low" cohesion when they have too many different types of responsibilities. Because of this:
 - Element becomes more difficult to understand
 - Reuse becomes more difficult
 - Maintenance becomes more complex



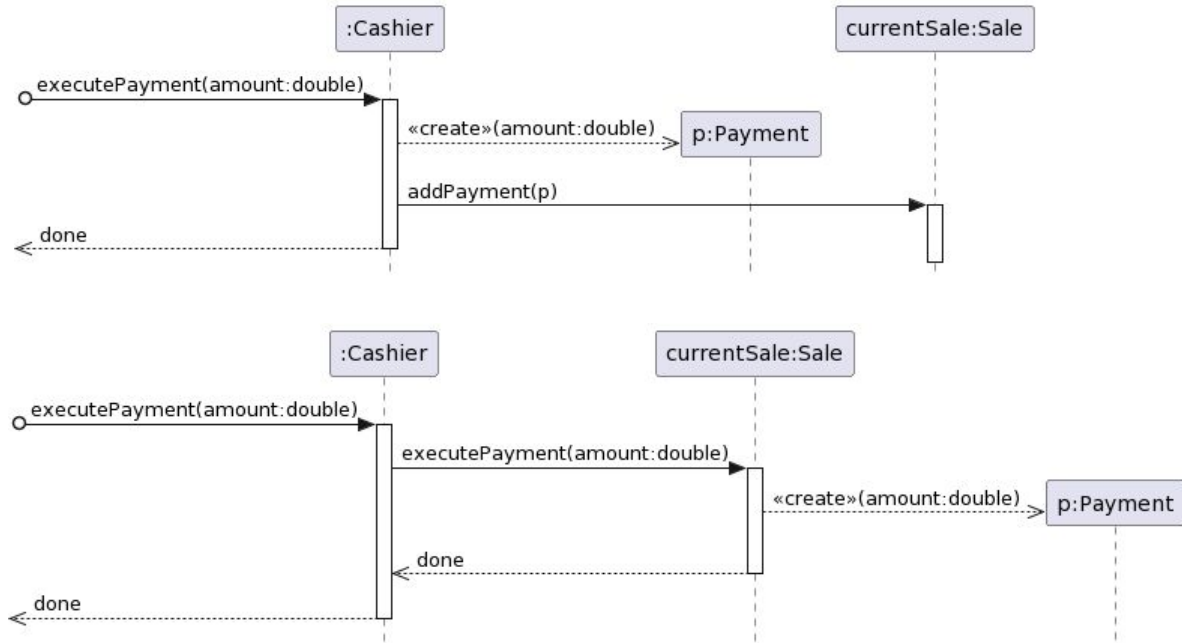
Principle: high cohesion

- Problem:
How do you keep objects focused, understandable and maintainable?
- Solution:
Assign responsibilities so that a class is a cohesive whole ("has high cohesion").



Example: high cohesion

- Example: which of the two systems has the highest cohesion and is preferred by the High Cohesion pattern?



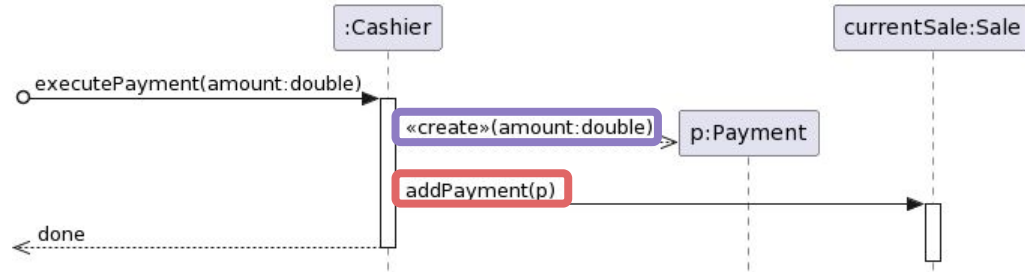
Example: high cohesion

- Example: which of the two systems has the highest cohesion and is preferred by the High Cohesion pattern?

Cohesion of Cashier class

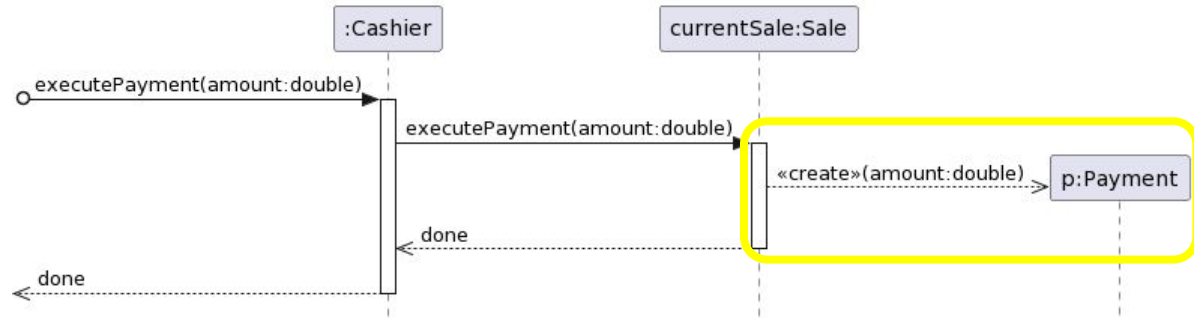
1. create p
2. add p to currentSale

Creating objects and then passing them on becomes problematic in a larger system



Cohesion of Cashier class

1. delegate to currentSale



The cohesion of Cashier at the bottom is higher, so better design according to this pattern!

Principle: high cohesion

- Points of interest:
 - A class with high cohesion:
 - Has limited clear and delineated responsibility
 - Usually has relatively few methods
 - Doesn't do too much work in itself
 - Works primarily with other objects
 - Compare with the real world: A bricklayer may be able to lay pipes but a plumber does it faster and better



Controller pattern

Pattern: controller

- Problem: Which object coordinates operation(s) and is addressed by the UI?
- Solution: assign this responsibility to a controller class (which has high cohesion: only coordination responsibilities).
- A controller allows us to keep input/output separate from processing. Separate layers.

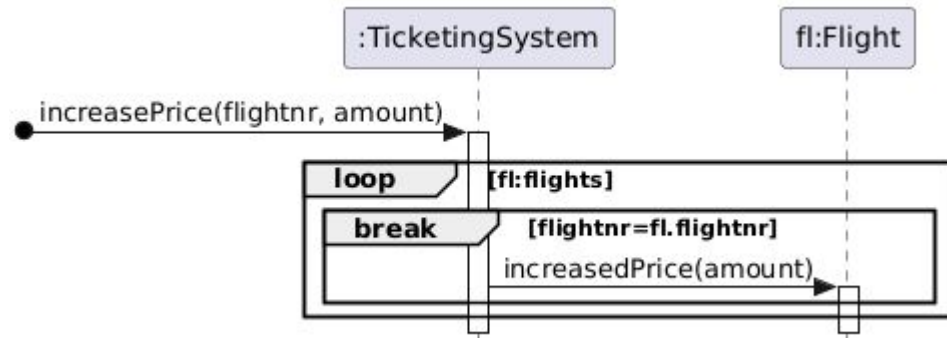
Controller types

- Two types of controllers:
 - Facade controller: An object that...
 - ... suggests the entire 'system'
 - ... represents a complete 'device' on which the software runs
 - ... proposes a complete 'major subsystem'
 - Use case/Session controller: An object that...
 - ... Delegates all events around a particular use case
 - ... Handles all interaction around a session with an actor

Example Façade controller

- Ruling: when do we consider something a facade controller?
 - It has as controller name the name of the system/company
 - For example, "ReservationSystem" in a restaurant, "POS" for a point-of-sale system
 - Its name is simply "Controller"

An example:



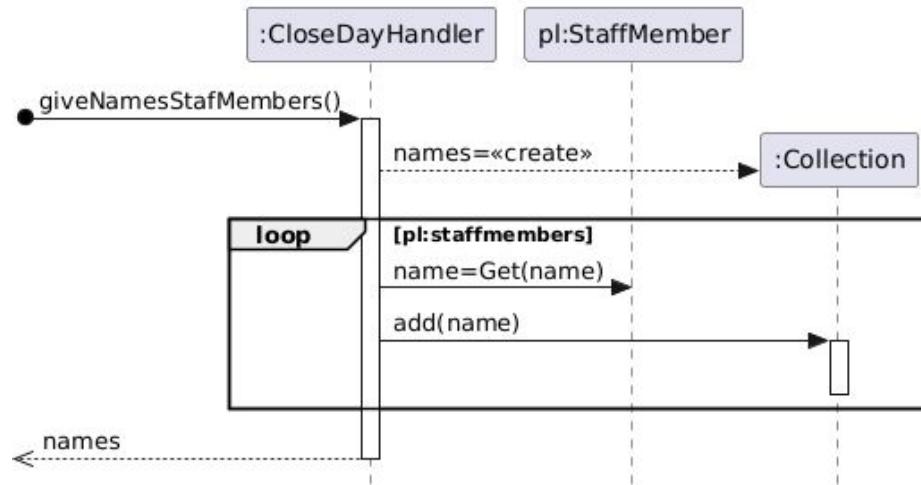
Example UC/session controller

- UC/session controllers for UC scenario defined
 - Commonly used designation are:
 - <UseCaseName>Controller
 - <UseCaseName>Handler
 - <UseCaseName>Coordinator
 - <UseCaseName>Session (Session is a conversation with an actor, here within the context/length of a UC execution)
 - For example:
 - Use case "Recording order": CustomerOrderHandler
 - Use case "Register reservation":
ReservationRegistreringController
 - Use case "Borrowing book": BookLenderCoordinator

Example UC/session controller

- Example: UC "Closing day" of a restaurant

All operations within this UC use the same controller:
CloseDayHandler



Controller delegates

- The controller delegates!
 - “Normally, a controller should delegate to other objects the work that needs to be done; it coordinates or controls the activity. It does not do much work itself.”
- Choose a Façade controller unless consistency is too degraded
 - If the system is limited, use a Façade controller. For larger system: split into separate controllers!

Controller not overdoing it

- Avoid a "bloated controller", such as:
 - Having only one controller handling all system events (operations in a UC): this is too low cohesion
 - A controller who performs many tasks himself instead of delegating them
 - A controller that itself has too much knowledge (attributes...) and thus keeps track of duplicate information
- How to prevent a bloated controller?
 - Use multiple controllers, e.g. one controller per UC instead of one facade controller for the whole system
 - Make sure the controller actually delegates rather than doing too many things himself

Creator pattern

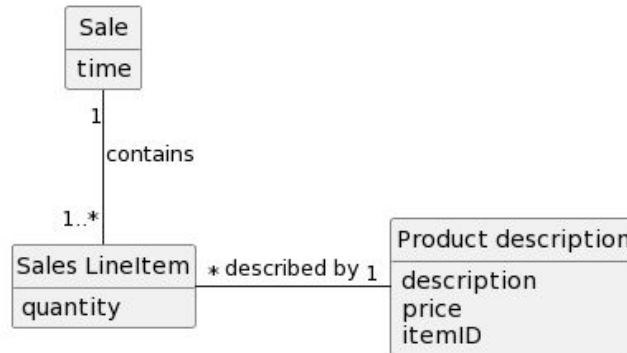


Pattern: creator

- Problem: Which object is responsible to create another object (A)?
- Solution:
 - Assign responsibility to an object (B) that is already linked (so linkage is not unnecessarily increased).
- So choose an object B that will create object A for which as many as possible of the statements below are true:
 - B "contains" A (composition): this is preferred in case of equal choice
 - B registers A
 - B makes strong use of A
 - B holds the data to initialise A

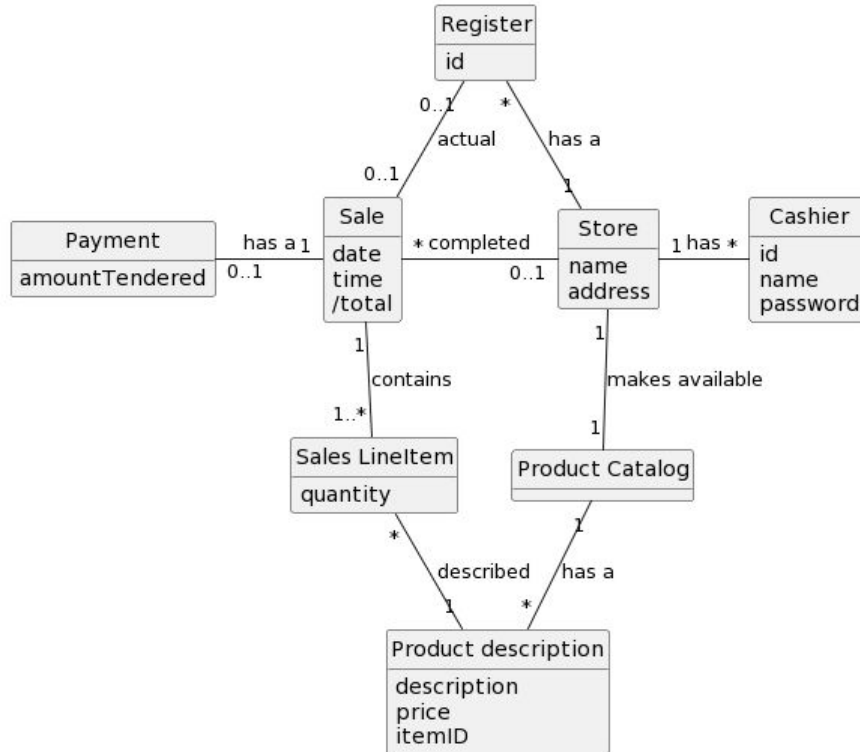
Example: creator

- Example: from which object do we create a SalesLineItem?
- We look at the propositions:
 - B "contains" A (composition): this is preferred in case of equal choice
 - B registers A
 - B makes strong use of A
 - B holds the data to initialise A
- Choice goes to objects of Sale because they contain SalesLineItems

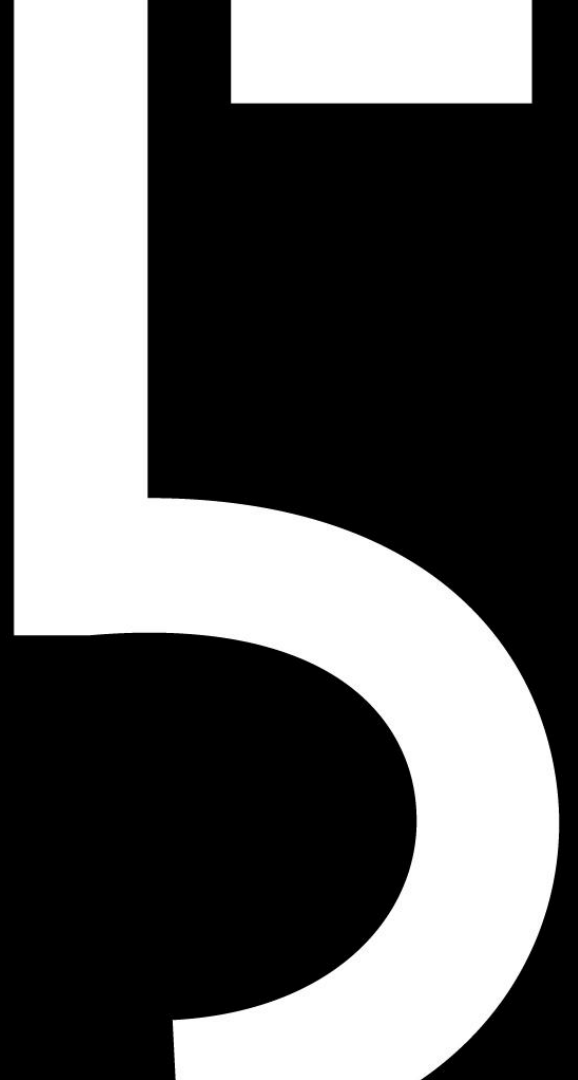


Example: creator 2

- Example: from which object do we create a ProductDescription?
- From a ProductCatalogue



Information expert pattern



Pattern Information Expert

- Problem:
 - What is the general principle of assigning responsibilities?
- Solution:
 - Assign responsibility to the class that has the necessary information to fulfil the responsibility.

Assign operations (the "responsibilities") to the class that has the attributes and references (the "information") to perform the operations.

Pattern Information Expert

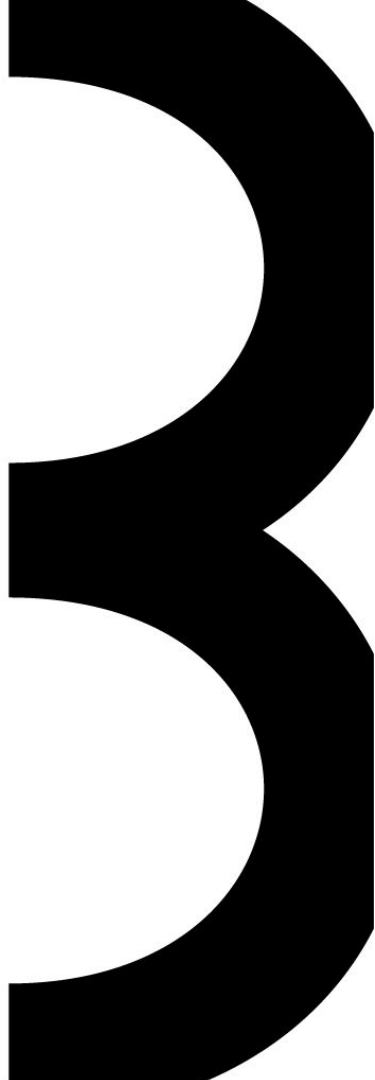
- Points of interest:
 - Look for candidates for an information expert class:
 - Is there a relevant class in the domain model?
 - If not, find a way to extend the model with a design class that can serve as an expert.
 - This is similar to real world:
 - Who is responsible for updating a database?
 - The one who has knowledge about the database!

Pattern Information Expert

- Points of interest:
 - Breaking down is sub-responsibilities
 - A responsibility sometimes has to be split into several sub-responsibilities to fit the pattern

Other patterns

(for reference, not the subject of this course)



5 SOLID principles

S

Single responsibility principle



O

Open closed principle



L

Liskov substitution principle



I

Interface segregation principle



D

Dependency inversion principle



SOLID focuses more on the quality and consistency of the code, while **GRASP** focuses more on the assignment and collaboration of the classes.

Gang Of Four (GoF) Patterns

2nd year!!!

GRASP focuses on higher-level design patterns, while the **GOF** patterns show technical implementation patterns



GoF Patterns Classification

4 More GRASP Patterns

There are **4 more GRASP patterns** that we will not discuss in this course (see more advanced courses)

